



VPC Endpoints



jjoseph1608@outlook.com

Successfully created VPC endpoint
vpce-02b57c8e1d5735e4b

Endpoints (1/1) Info

Find endpoints by attribute or tag

VPC endpoint ID: vpce-02b57c8e1d5735e4b

Clear filters

< 1 >

☒

Name

☐

VPC endpoint ID

☐

Endpoint type

☐

Status

☐

Service

vpce-02b57c8e1d5735e4b / NextWork VPC Endpoint

Details

Route tables

Policy

Tags

Details

Endpoint ID
vpce-02b57c8e1d5735e4b

VPC ID
vpc-0e0ba9e948e067300 (NextWork-vpc)

DNS record IP type
service-defined

Private DNS specified domains
-

Status
Available

Status message
-

IP address type
ipv4

Creation time
Friday, 5 June 2026 at 16:47:19 BST

Service name
com.amazonaws.us-east-1.s3

Service region
us-east-1

Endpoint type
Gateway

Private DNS names enabled
No

Private DNS preference
-

A blue circular logo with the letters 'JJ' in white.

Introducing Today's Project!

What is Amazon VPC?

"Amazon VPC is a service that lets you create isolated private networks within AWS, and it is useful because it gives you full control over your network environment including IP ranges, subnets, route tables, and security settings, keeping your cloud resources separate and secure

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create a private network, launch an EC2 instance into it, and set up a VPC endpoint to give that instance direct private access to my S3 bucket without any traffic travelling over the public internet

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how immediately the bucket policy started blocking access after I applied it. I wasn't prepared for the S3 console itself to show denied access warnings, which made me realise that the policy was working so well it was even blocking my own browser traffic because it wasn't coming through the endpoint

This project took me...

This project took me around 35 minutes to complete, including setting up the VPC, launching the EC2 instance, configuring access keys, creating the S3 bucket, setting up the VPC endpoint, creating the bucket policy, updating the route table, and testing the connection



In the first part of my project...

Step 1 - Architecture set up

In this step, I will create a VPC, launch an EC2 instance, and set up an S3 bucket, because these are the foundational resources I need before I can set up a VPC endpoint and test private connectivity between my instance and my bucket

Step 2 - Connect to EC2 instance

In this step, I will connect to my EC2 instance using EC2 Instance Connect, because I need to get inside the instance and use the command line to configure my AWS credentials and start testing connectivity to my S3 bucket through the VPC endpoint

Step 3 - Set up access keys

In this step, I will create access keys and configure them on my EC2 instance using the AWS CLI, because without valid credentials my instance has no way to authenticate requests to AWS and interact with services like Amazon S3

Step 4 - Interact with S3 bucket

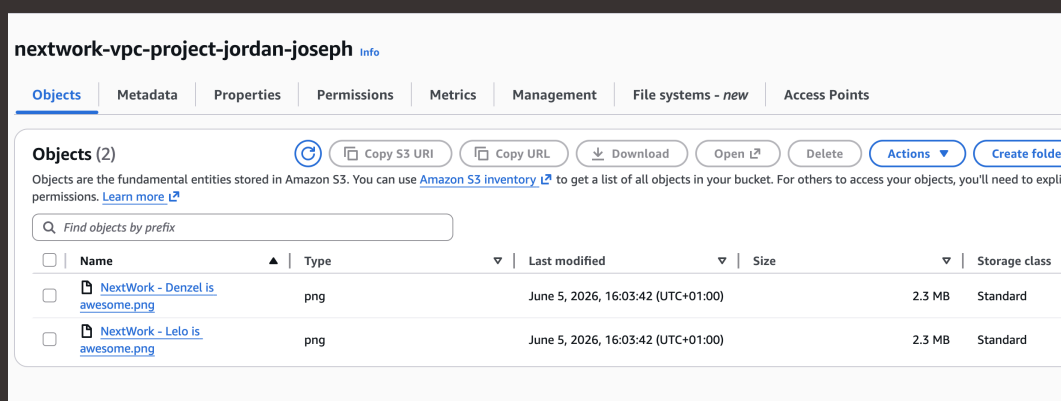
In this step, I will head back to my EC2 instance and run AWS CLI commands to access my S3 bucket, because now that my credentials are configured I can test whether my instance can successfully communicate with Amazon S3



Architecture set up

I started my project by launching a VPC with a public subnet, an internet gateway, and a route table, then launched an EC2 instance into the VPC

I also set up an S3 bucket that will act as the target for my VPC endpoint, giving me a private storage destination that my EC2 instance will connect to without traffic ever leaving the AWS network





Access keys

Credentials

To set up my EC2 instance to interact with my AWS environment, I configured my access key ID, secret access key, default region, and default output format using the `aws configure` command, which stores these credentials on the instance so that every AWS CLI command I run is authenticated against my AWS account

Access keys are a pair of credentials made up of an access key ID and a secret access key that are used to authenticate programmatic requests to AWS, allowing tools like the AWS CLI to interact with your AWS account and its services on your behalf

Secret access keys are the private part of an access key pair that acts like a password, used alongside the access key ID to authenticate requests to AWS. They should be kept secure and never shared, as anyone with access to them can make requests to your AWS account on your behalf

Best practice

Although I'm using access keys in this project, a best practice alternative is to use IAM roles attached directly to the EC2 instance, because this avoids storing long term credentials on the instance and instead lets AWS handle authentication automatically and securely



Connecting to my S3 bucket

The command I ran was `aws s3 ls`. This command is used to list all of the S3 buckets in my AWS account, and I ran it to test whether my EC2 instance had the necessary permissions to communicate with Amazon S3

The terminal responded with a list of my S3 buckets. This indicated that the access keys I set up were working correctly and my EC2 instance was successfully authenticated and able to communicate with Amazon S3

```
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls
2026-06-05 14:19:35 nextwork-vpc-project-jordan-joseph
[ec2-user@ip-10-0-13-76 ~]$
```



Connecting to my S3 bucket

I also tested the command `aws s3 ls s3://nextwork-vpc-project-jordan-joseph` which returned a list of the objects stored inside that specific bucket, confirming that my EC2 instance had the permissions needed to access and read the contents of my S3 bucket

```
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls
2026-06-05 14:19:35 nextwork-vpc-project-jordan-joseph
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
[ec2-user@ip-10-0-13-76 ~]$
```



Uploading objects to S3

To upload a new file to my bucket, I first ran the command `sudo touch /tmp/test.txt`. This command creates a new empty file called `test.txt` in the `tmp` directory on my EC2 instance, giving me something to upload to my S3 bucket

The second command I ran was `aws s3 cp /tmp/test.txt s3://nextwork-vpc-project-jordan-joseph`. This command will copy the `test.txt` file from my EC2 instance and upload it to my S3 bucket, testing that my instance has the correct permissions to write objects to Amazon S3

The third command I ran was `aws s3 ls s3://nextwork-vpc-project-jordan-joseph` which validated that the `test.txt` file had been successfully uploaded to my S3 bucket by showing it appearing in the list of objects stored inside it

```
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls
2026-06-05 14:19:35 nextwork-vpc-project-jordan-joseph
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
[ec2-user@ip-10-0-13-76 ~]$ sudo touch /tmp/nextwork.txt
[ec2-user@ip-10-0-13-76 ~]$ aws s3 cp /tmp/nextwork.txt s3://nextwork-vpc-project-jordan-joseph
upload: ../../tmp/nextwork.txt to s3://nextwork-vpc-project-jordan-joseph/nextwork.txt
[ec2-user@ip-10-0-13-76 ~]$ aws s3 ls s3://nextwork-vpc-project-jordan-joseph
2026-06-05 15:03:42      2431554 NextWork - Denzel is awesome.png
2026-06-05 15:03:42      2399812 NextWork - Lelo is awesome.png
2026-06-05 15:41:31           0 nextwork.txt
[ec2-user@ip-10-0-13-76 ~]$
```



In the second part of my project...

Step 5 - Set up a Gateway

In this step, I will create a VPC endpoint for Amazon S3, because this gives my EC2 instance a direct private route to my S3 bucket without any traffic needing to leave the AWS network and travel over the public internet

Step 6 - Bucket policies

In this step, I will update my S3 bucket policy to restrict access so that only traffic coming through my VPC endpoint is allowed, because this ensures that my bucket cannot be accessed from anywhere outside my private VPC connection

Step 7 - Update route tables

"In this step, I will test my VPC endpoint setup by running AWS CLI commands from my EC2 instance and troubleshoot any connectivity issues that come up, because this is how I confirm that the endpoint and bucket policy are working correctly together

Step 8 - Validate endpoint connection

In this step, I will test my VPC endpoint setup again to confirm it is working correctly, and then restrict my VPC's access to my AWS environment so that only authorised traffic can interact with my AWS account

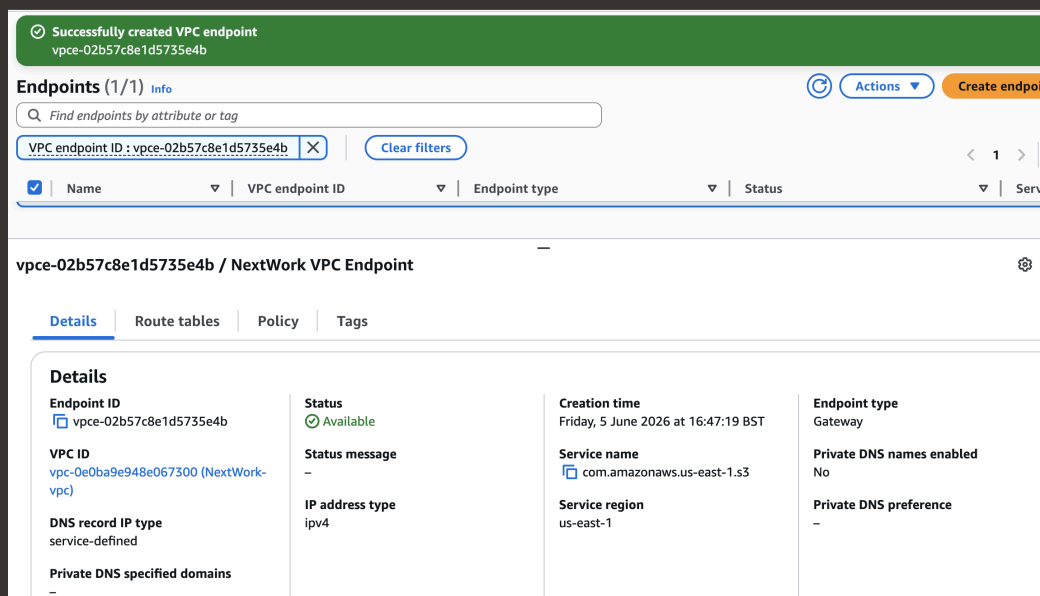


Setting up a Gateway

I set up an S3 Gateway, which is a type of VPC endpoint specifically used to connect a VPC directly to Amazon S3 or DynamoDB. It works by adding a route to your VPC route table that points S3 traffic through the gateway instead of out to the public internet

What are endpoints?

An endpoint is a private connection between your VPC and an AWS service that allows traffic to flow directly between them without going over the public internet, keeping communication faster, more secure, and entirely within the AWS network





Bucket policies

A bucket policy is a resource based policy attached directly to an S3 bucket that defines who is allowed to access it and what actions they can perform, such as reading or writing objects, and can be used to restrict access to specific users, roles, or in this case a specific VPC endpoint

My bucket policy will deny all requests to my S3 bucket that do not come through my VPC endpoint, meaning that even if someone has valid AWS credentials they will not be able to access the bucket unless their traffic is routing through the endpoint

Policy

```
1  {
2    "Version": "2012-10-17",
3    "Statement": [
4      {
5        "Effect": "Deny",
6        "Principal": "*",
7        "Action": "s3:*",
8        "Resource": [
9          "arn:aws:s3:::nextwork-vpc-project-jordan-joseph",
10         "arn:aws:s3:::nextwork-vpc-project-jordan-joseph/*"
11       ],
12       "Condition": {
13         "StringNotEquals": {
14           "aws:sourceVpce": "vpce-02b57c8e1d5735e4b"
15         }
16       }
17     ]
18   }
19 }
20
```



Bucket policies

Right after saving my bucket policy, my S3 bucket page showed denied access warnings. This was because the policy I applied restricts access to only traffic coming through the VPC endpoint, and my browser was accessing the S3 console over the public internet rather than through the endpoint, so it was being blocked by the policy

I also had to update my route table because without a route pointing S3 traffic through the VPC endpoint, my EC2 instance would still try to reach S3 over the public internet and the bucket policy would block it. Adding the endpoint route tells my VPC to send all S3 traffic through the private gateway instead

Block public access (bucket settings)

Edit

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to all your S3 buckets and objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to your buckets or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

Access deniedDiagnose with Amazon Q

API response

User: arn:aws:iam::304152263329:user/Jordan is not authorized to perform: s3:GetBucketPublicAccessBlock on resource: "arn:aws:s3:::nextwork-vpc-project-jordan-joseph" with an explicit deny in a resource-based policy

Bucket policy

EditDelete

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

You don't have permission to get bucket policyDiagnose with Amazon Q

You or your AWS administrator must update your IAM permissions to allow s3:GetBucketPolicy. After you obtain the necessary permission, refresh the page. Learn more about [Identity and access management in Amazon S3](#)

API response



Route table updates

To update my route table, I edited it to associate my VPC endpoint with it, which automatically added a route that directs all S3 traffic from my VPC through the gateway endpoint instead of out to the public internet

After updating my public subnet's route table, my terminal could return a list of objects in my S3 bucket when I ran the `aws s3 ls` command, confirming that my EC2 instance was now successfully routing S3 traffic through the VPC endpoint and the bucket policy was allowing it through

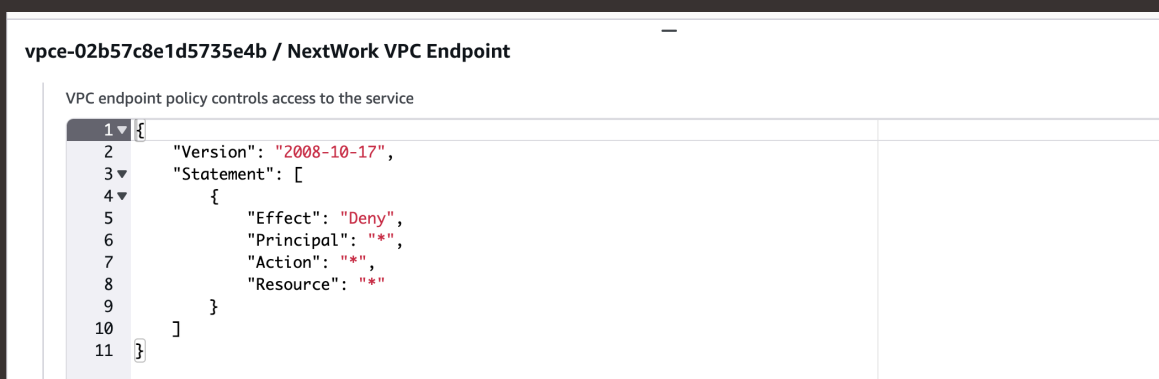
Details	Flow logs	Route table	Network ACL	CIDR reservations	Sharing	Tags
Route table: rtb-0b7d28660ce677a5e / NextWork-rtb-public Edit route table						
Routes (3)						
Filter routes						
Destination		Target				
10.0.0.0/16		local				
0.0.0.0/0		igw-006aac414e6c2bd3f				
pl-63a5400a		vpce-02b57c8e1d5735e4b				



Endpoint policies

An endpoint policy is a resource based policy attached to a VPC endpoint that controls which AWS services, actions, and resources can be accessed through that endpoint, giving you an additional layer of control over what traffic is allowed to flow through the private connection

I updated my endpoint's policy by restricting it to only allow access to my specific S3 bucket, blocking access to any other S3 buckets in my account. I could see the effect of this right away, because when I ran the `aws s3 ls` command to list all my buckets it returned an access denied error, confirming that the endpoint policy was successfully limiting what my EC2 instance could access through the endpoint





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

